

STRAVYX

Australia's On-Demand Drone Services Network

DJI Frontend Technical Spec - Realtime & Native Bridge

packages/realtime WS contract | packages/dji-msdk native bridge | shared types

Stravix Pty Ltd ACN 696 964 271 | Jul 2026 | CONFIDENTIAL

Source: docs/dji-frontend-technical-spec.md. The level below dji-frontend-integration.md: wire contracts, TypeScript interfaces and native module signatures for the two hard dependencies - packages/realtime (WebSocket) and packages/dji-msdk (native MSDK bridge). Proposed contracts (Phase 1+); lock field names against the NestJS visibility module and live DJI payloads at build.

0. System Context - Three Hops

DJI devices never talk to the browser directly. Flow: device -> DJI gateway (Pilot 2 / Dock) -> NestJS Cloud API bridge -> WS gateway (packages/realtime) -> client (web / mobile).

Two golden rules

- The bridge NORMALIZES raw DJI thing-model payloads into stable Stravix types (client insulated from DJI schema/version churn).
- The bridge FILTERS by role before publishing - operators/pilots/dock owners never receive customerTotal or Layer 2 fields, even in telemetry envelopes.

1. Shared Types (packages/types)

Single source of truth imported by web, mobile, realtime, dji-msdk, and api-client.

```
export type MissionStatus =
  | 'DRAFT' | 'SUBMITTED' | 'QUOTING' | 'DISPATCHING' | 'CONFIRMED'
  | 'READY' | 'AIRBORNE' | 'COMPLETE' | 'IN_REVIEW' | 'CLOSED' | 'DISPUTED';

export interface Telemetry {
  missionId: string; deviceSn: string;
  position: GeoPoint; headingDeg: number; speedMs: number;
  batteryPct: number; // never exposes raw cell data
  gimbalPitchDeg?: number; homeDistanceM?: number;
  capturedAt: string; // ISO-8601, bridge-stamped
  seq: number; // monotonic per mission for ordering
}

export interface UploadProgress {
  missionId: string; fileId: string; fileName: string;
  kind: 'raw' | 'preview'; bytesUploaded: number; bytesTotal: number;
  state: 'queued' | 'uploading' | 'done' | 'failed';
  fingerprint?: string; // chain-of-custody / dispute audit
}

export interface HmsAlert {
  deviceSn: string; missionId?: string; code: string;
  severity: 'info' | 'warn' | 'critical'; message: string; raisedAt: string;
  blocksFlight: boolean; // drives READY->AIRBORNE gate + admin freeze
}

export interface Deliverable {
  missionId: string; id: string; layer: 1 | 2;
  type: 'photo' | 'video' | 'orthomosaic' | 'model3d' | 'gaussian' | 'pointcloud';
  status: 'queued' | 'processing' | 'ready' | 'failed';
  url?: string; thumbnailUrl?: string; shareToken?: string;
}
```

Visibility note: these are the MAXIMAL shapes. The bridge emits role-projected subsets (sec 2.4). Layer 2 cost fields do not exist on the type at all - margin data lives only in admin-only server DTOs.

2. packages/realtime - WebSocket Contract

2.1 Connection & auth

Item	Value
Endpoint	wss://api.stravyx.com/ws (ap-southeast-2)
Auth	Bearer JWT on upgrade (Auth0/Cognito); role claim drives filtering
Heartbeat	client ping 20s; server closes stale after 45s
Reconnect	exponential backoff 1s->30s + jitter; resubscribe on open
Ordering	per-subscription seq; client drops out-of-order telemetry
Delivery	snapshot then deltas - first message on subscribe is a full snapshot

2.2 Message envelope (discriminated union on type)

```
export type ServerFrame =
  | { type: 'snapshot'; channel: Channel; id: string; data: SnapshotPayload }
  | { type: 'telemetry'; id: string; seq: number; data: Telemetry }
  | { type: 'mission_status'; id: string; data: { status: MissionStatus; at: string } }
  | { type: 'mission_event'; id: string; data: MissionEvent }
  | { type: 'upload_progress'; id: string; data: UploadProgress }
  | { type: 'hms'; id: string; data: HmsAlert }
  | { type: 'fleet'; data: FleetSnapshot } // admin only
  | { type: 'error'; code: string; message: string }
  | { type: 'pong'; t: number };
```

2.3 Bridge mapping - DJI MQTT -> Stravyx WS

DJI MQTT topic	Bridge action	Emitted frame
thing/product/{sn}/osd	throttle ~1-2 Hz, project fields	telemetry
thing/product/{sn}/state	detect status-relevant changes	mission_status / mission_event
thing/product/{sn}/events (upload)	map progress	upload_progress
thing/product/{sn}/events (HMS)	severity + blocksFlight mapping	hms
sys/product/{gw}/status (update_topo)	online/offline + capability	fleet (admin)

Command direction (start mission, RTH, dock control) goes via HTTPS/services topics - plain REST in api-client, not WS (sec 4).

2.4 Role projection (enforced server-side)

Frame	consumer	mobile_operator	admin
telemetry	position, status, batteryPct	full (own mission)	full
upload_progress	high-level % only	full (own files)	full
hms	none	own device	all
fleet	-	-	full
L2 Deliverable	yes (when ready)	never	yes

2.5 React hooks (web + RN identical surface)

```
export function useMissionStream(missionId: string): {
  status: MissionStatus; telemetry?: Telemetry;
  uploads: UploadProgress[]; hms: HmsAlert[]; connected: boolean;
};
export function useFleetStream(): { devices: FleetDevice[]; connected: boolean }; // admin
// One socket, multiplexed across subscriptions, wrapped by RealtimeProvider.
```

3. packages/dji-msdk - Native Bridge (mobile only)

Wraps MSDK v5 (Android Kotlin AAR / iOS Swift framework) behind one TS surface. Consumed by operator-mobile (full); consumer-mobile uses realtime instead.

3.1 Module surface

```
export interface DjimSdk {
  // lifecycle
  registerApp(): Promise<void>;
  connectProduct(): Promise<ProductInfo>;
  getProductInfo(): Promise<ProductInfo | null>;
  // missions (IWaypointMissionManager / IWPMPZManager)
  loadWaypointMission(kmzUri: string): Promise<{ missionId: string }>;
  startWaypointMission(): Promise<void>;
  pauseWaypointMission(): Promise<void>;
  resumeWaypointMission(): Promise<void>;
  stopWaypointMission(): Promise<void>;
  // livestream (ILiveStreamManager)
  startLiveStream(cfg: { url: string; protocol: 'rtmp'|'rtsp'; bitrateKbps?: number }): Promise<void>;
  stopLiveStream(): Promise<void>;
  // media (IMediaManager)
  listMedia(): Promise<MediaFile[]>;
  uploadMedia(o: { fileIds: string[]; sts: StsCredentials; missionId: string }): Promise<void>;
  // safety
  getHmsState(): Promise<HmsAlert[]>;
  startReturnToHome(): Promise<void>;
}
```

3.2 Event emitter (native -> JS, mirrors WS frames)

```
export type Djievent =
  | { type: 'connection'; data: { rcConnected: boolean; aircraftConnected: boolean } }
  | { type: 'telemetry'; data: Telemetry } // same packages/types shape
  | { type: 'missionState'; data: { status: MissionStatus; waypointIndex?: number } }
  | { type: 'uploadProgress'; data: UploadProgress }
  | { type: 'hms'; data: HmsAlert }
  | { type: 'error'; data: DjieError };
export function addDjiListener(cb: (e: Djievent) => void): () => void;
```

Design symmetry: Telemetry/UploadProgress/HmsAlert are the SAME types the bridge emits, so FlightHud binds to either useMissionStream() (cloud path) or addDjiListener() (direct SDK path) via one adapter - no duplicate UI.

3.4 Native mapping

TS method / event	Android (MSDK v5)	iOS (MSDK v5)
startWaypointMission	IWaypointMissionManager.startMission	WaypointMissionOperator.startMission
missionState event	WaypointMissionExecuteStateListener	mission operator listener
startLiveStream	ILiveStreamManager.startStream	DJILiveStreamManager
listMedia / uploadMedia	IMediaManager + S3 multipart	MediaManager + S3 multipart
telemetry event	FC/battery/gimbal key listeners	key manager listeners

3.5 Lifecycle & threading

- registerApp() once at app start (needs network for first activation); cache activation.
- SDK callbacks arrive on native threads -> bridge marshals to JS thread before emitting.
- RC link is source of truth: disable Fly/Stream/Upload UI unless rcConnected && aircraftConnected.
- Foreground service (Android) / background modes (iOS) keep telemetry + upload alive when screen locks mid-mission.

4. Command Path (packages/api-client) - Not WebSocket

Outbound actions are REST (idempotent, auditable), not WS.

```
export const djiApi = {
  dispatchWayline: (missionId, kmzId) => post(`/missions/${missionId}/dispatch`, { kmzId }),
  getStsCredentials: (missionId) => post(`/missions/${missionId}/upload-credentials`, {}),
  dockCommand: (dockId, cmd /* open|close|start_task|stop_task|force_land */) =>
    post(`/docks/${dockId}/command`, { cmd }), // admin only; server writes audit (M-07)
};
```

Every dockCommand requires a client confirm modal AND server-side RBAC + audit. The UI never assumes success; it awaits the resulting mission_status / fleet WS frame.

5. Media Viewer Inputs (packages/media)

FlightHub 2 Modeling API outputs map to viewer props:

```
OrthoViewerProps    { url; boundsGeoJson? } // 2D orthomosaic (COG/tiles)
Model3DViewerProps  { url; format: 'glb'|'obj'|'3dtiles' } // photogrammetry mesh
PointCloudViewerProps { url; format: 'las'|'laz'|'potree' } // LiDAR
GaussianViewerProps { url } // 3DGS splat
```

DeliverableGallery switches viewer by Deliverable.type; all sources are signed short-TTL URLs issued by api-client (never public buckets).

6. Test & Acceptance Hooks

Contract	Test
Role projection	Contract tests: consumer/operator WS payloads contain no L2 or customerTotal fields
Ordering	Inject out-of-order seq; client renders latest only
Reconnect	Kill socket mid-mission; snapshot re-hydrates tracker within 2s
Bridge normalization	Replay recorded DJI OSD fixtures -> assert stable Telemetry
dji-msdk parity	Same DjiiEvent stream shape on Android + iOS simulators (mock SDK)
HMS gate	blocksFlight:true disables startWaypointMission + blocks READY->AIRBORNE

Markdown source: docs/dji-frontend-technical-spec.md. Related: dji-frontend-integration.md, dji-integration-catalogue.md, mvp-build-timeline.md. All interfaces are proposed contracts - lock at build.